Comparative Analysis of Proton Flux Forecasting with a Time Series Transformer

Lewis Adkins[1]

[1]Space Weather Lab Group, George Mason University

Key points

- TBD
- TBD
- TBD

1 Abstract

Solar storms are able to cause damage to vulnerable space and ground based platform components. These solar storms consists of many classes of particles such as electrons, protons, alpha particles, heavy nuclei and neutrons. Particles that cause the most damage to equipment are protons > 10 MeV colliding with satellites and discharging above the critical charge, Q_c, to cause single event upsets (SEUs). An SEU is the process in which a bit flip occurs in the static random access memory unit causing anomalies in data or soft errors. To safeguard against these events we predict solar storms from the Comprehensive Suprathermal and Energetic Particle Analyzer - Electron Proton Helium Instrument (COSTEP-EPHIN) on the Solar and Heliospheric Observatory (SOHO) particle flux channels. Two current relativistic electron intensity channels ($> .25$ MeV and > 0.67 MeV) and proton fluxes from solar storms to forecast future proton intensity levels. The dataset spans from December of 1995 to January of 2002 with a 5 minute cadence. Using particle flux to predict was previously attempted using a forecasting matrix mapping the Electron Increase Parameter to the electron intensity which would give a heatmap of the proton intensity. With the advent of machine learning, Neural Networks and Recurrent Neural Networks provided better forecasting F1 values than it predecessors. We choose a collection of transformer architectures to evaluate against each previous models and compare against Mean Absolute Error (MAE), lag from onset to peak fluxes, and F1 scores. We demonstrate a reduction in the MAE across all transformer architectures with similar lag values. On average we find improved F1 scores marking a well designed forecasting model. This paper will demonstrate a baseline case to make related predictions on other satellite platforms such as GOES.

2 Introduction

Space based equipment and platforms are vulnerable to their surrounding Space Weather Environment (SWE) . The Sun dominates the SWE with solar storms comprised of winds that travel from its surface to us. The storms themselves are made from a composite of classes of particles such as protons, electrons, alpha particles, and heavier nucleons. Together, because of their origination, they make up what we call Solar Energetic Particles (SEP)s. The SEP's travel from our sun and collide with our satellites and equipment resulting in damage done to the Static Random Access Memory (SRAM) of their components (Noeldeke et al., 2021). Because of their abundance and difficulty in shielding, high energy protons (> 10 MeV) are the largest culprit of damaging our equipment. (Torres et al., 2025). The protons cause "soft" anomalies in the Static Random Access Memory (SRAM) resulting in bit flips in the telemetry data (Carlton et al., 2018) if the energy of the proton is larger than the critical charge, Q_c, of the equipment (Barak et al., 1999).

Relativistic electrons will typically arrive 1 AU ahead of non-relativistic high energy protons due to the less scattering, making them a hallmark for predicting solar storms (Posner, 2007). This relationship was demonstrated by (Van Hollebeke et al., 1975) ← can't access, who describing electrons as the first in situ sign of a particle event. A forecasting matrix method mapping the electron increase parameter to the electron intensity creating a heat map for the average upcoming proton intensities for each electron rise-electron flux pair combination (Posner, 2007). Machine learning algorithms were later developed and an Neural Network (NN) and Recurrent Neural Network (RNN) was able to achieve a higher forecasting F1 score then its forecasting matrix predecessor (Torres et al., 2025).

In this paper, we test if a time series transformer is able to attain improved forecasting scores than the previously stated methods? We examine multiple positional encoder architectures to create these predictions.

Section 3 will discuss related works replacing forecasting RNN models with transformers. Section 4 will briefly introduce the RNN model we will be replicating along with the chosen positional encoding methods to compare forecasting accuracy. We measure our models outputs with a persistence model as a baseline (Torres et al., 2025) and compare our error, lag and forecasting. Section 5 will compare models and show that we have

improved error reduction compared to (Torres et al., 2025) and (Posner, 2007) but similar lag and forecasting. Section 7 will summarize and discuss future work, potential improvements and limitations.

3 Related Work

There have been multiple studies of comparing other machine learning models with a time series transformer. One study forecasts future traffic conditions such as the density of vehicles and their speed to enable the prediction of futrre events like congestion and travel duration. This is all done by analyzing historical traffic conditions and patterns. Models based on RNN and Gated Recurrent Units GRU and Long Short Term Memory (LSTM) could effectively capture temporal dependencies sequationaly however but suffer from inherent deficienies. One is preserving very long term sequational information leading to loss of long term temporal dependency in time series. RNNs are not able to capture the periodicity of time series and treats all time steps equally. Short term data processing is unaffected but over the course of week... (Cai et al., 2020)

4 Approach

We propose seven time series transformers of different architectures to forecast proton intensities from preceding relativistic electrons. A transformer is a deep learning architecture that was first introduced in 2017 (Vaswani et al., 2017) as an improvement to previous machine learning models. We test this by using a window of electron and proton intensities to make future proton predictions. Current transformer models have been appealing as to researchers as they have the ability to capture long range dependencies and scale effectively with large dataset. This makes them suitable for our challenge of time series forecasting with multi horizon demand predictions.(Zhao et al., 2026). We make a selection of transformer models of different architectures to model our incoming proton intensity flux.

Section 4.1 will discuss the time time series dates and characteristics. Section **??** outline our transformer approach and compare the architectures with (Torres et al., 2025). In Section 4.3, we will explain each of the used architectures in our study.

4.1 Input

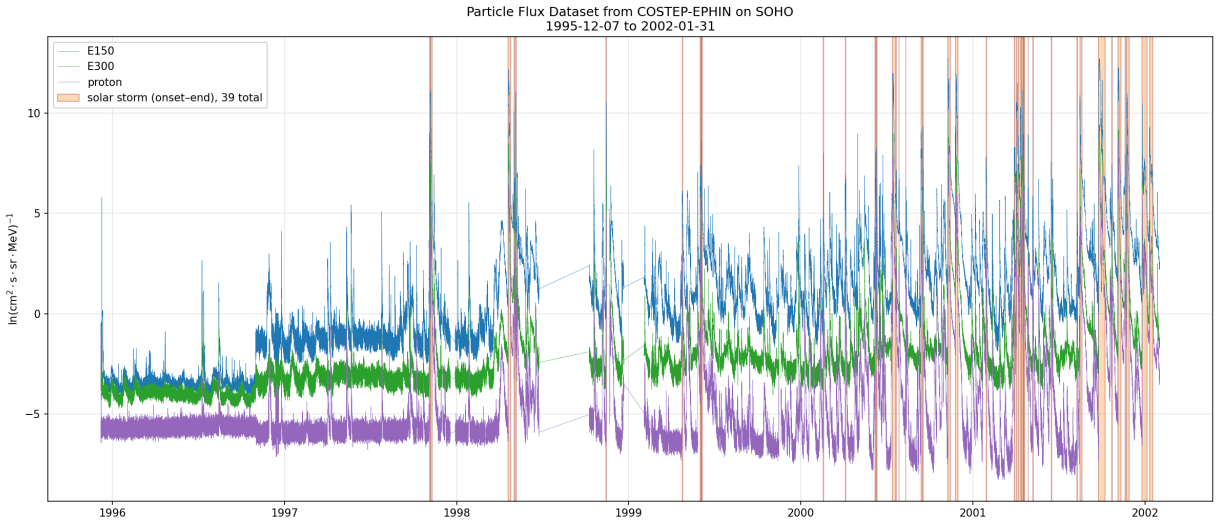


Figure 1. SOHO COSTEP-EPHIN dataset from 1995-2002

The input data is obtained from the Comprehensive Suprathermal and Energetic Particle Analyzer - Electron Proton Helium Instrument (COSTEP-EPHIN) on the Solar and Heliospheric Observatory (SOHO) located at Lagrange Point 1 (L1) (*SOHO Com-*

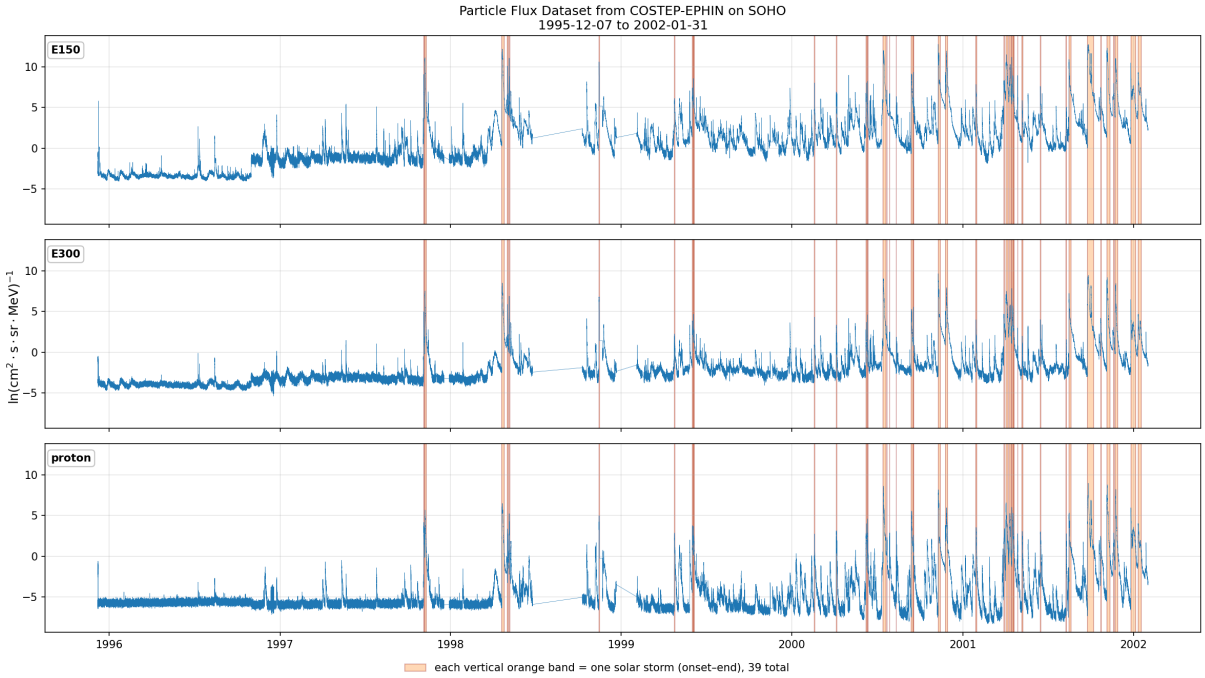


Figure 2. SOHO COSTEP-EPHIN dataset from 1995-2002

prehensive Suprathermal and Energetic Particle Analyzer (COSTEP) Electron Proton Helium Instrument (EPHIN) Intensity Level 3 1-minute Data in CDF Format, 2026). The main features we will be using to make are proton flux predictions are two electron intensities channels $\geq$0.25 and $\geq$0.67 MeV and the current proton intensity channel $\geq$10 MeV (Müller-Mellin et al., 1995). In total, the number of timestamped fluxes is $\approx 600,000$ entries. The resultant proton channel has been integrated across several channels and interpolated using the power-law spectrum. Each channel corresponds to a range of proton energies. Proton channel P4 corresponds to the energy range 4.3−7.8 MeV, P8 corresponds with $7.8-25-7.8$ MeV, P25 corresponds with $25-40.9$ MeV, and P41 corresponds with $40.9-53-7.8$ MeV A peak detection algorithm then sliced each SEP event for storms larger than 10 proton flux unit ,(pfu), and timestamped each phase within storm: Onset, threshold, peak, and end. The onset is defined when the flux rises three times the error bar above the pre-event background, the threshold is when the flux first crosses over 10 pfu and the end time is when it drops again below 2 pfu.(Torres et al., 2025). Each timestamp in our event catalog is given a bool indicating which phase the flux is in. The proton flux projections are calculated by a 2 hour window of the two elec-

tron channels and the proton fluxes and then casted out by both 30 minutes (t+6) and 1 hour ($t + 12$).

4.2 Transformers

Recurrent Neural Networks (RNN)s are unreliable for long range forecasting and irregular patterns (Zhao et al., 2026). This is prevalent in our particle flux data with long periods of noise with sudden flux spikes when a storm occurs. The transformer architecture allows for capturing long patterns of behavior with improved predictions with parallel processing while RNNs process only in series. The Transformer attention mechanism is able to analyze all time steps at once and focus on relevant steps regardless of order (Vaswani et al., 2017). This method of parallel analytical processing reveals subtle patterns that would be overlooked by RNNs (Zhao et al., 2026). Our transformer will only have an encoding only architecture as we are forecasting one value of proton flux, therefore a single output layer is required only. If we were to predict a series proton fluxes then a decoder would be required.

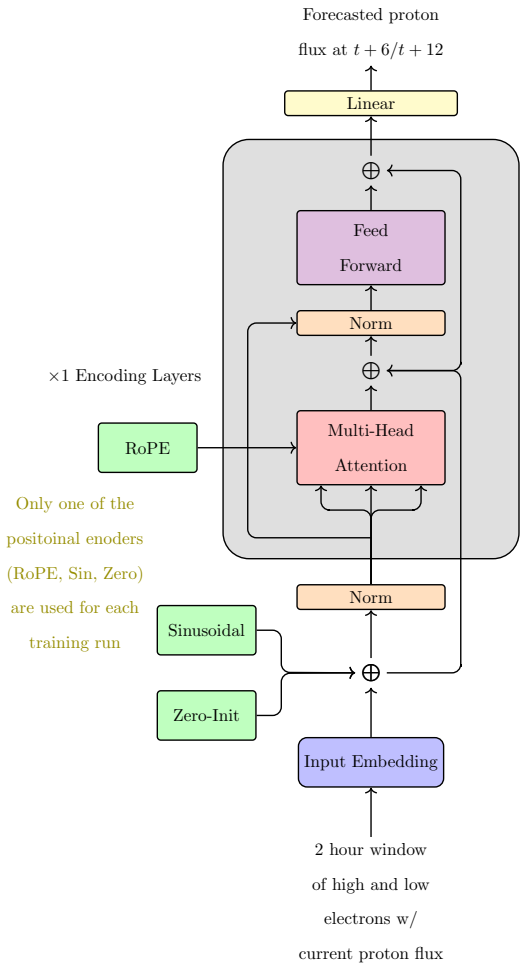


Figure 3. M1- Transformer Architecture with positional encodings

We want to avoid creating transformers that are larger than the provided models by (Torres et al., 2025) for our comparative analysis as it would lead to false improved scores. We measure the size of the models by counting the number of parameters within it. Parameters are the weights and biases inside a model whose values are learned form the trained data. If we increase the the number of parameters, the model is able to learn more complex relationships but with computation cost and the risk of overfitting. The RNN and NN are implemented with two layers: One layer is hidden with 30 units and then a second dense single output layer. The RNN is comprised of Gated Recurrent Units (GRU) for its hidden layers while the NN uses hidden layers. The NN model uses the same three channels for 25 timestep intervals. The RNN has the 30 GRU units plus our three channel input (Torres et al., 2025).

Models	Parameters
NN	2,311
RNN	3,181
Sinusoidal	3,361
Zero Init	3,761
RoPE	3,361

Figure 4. Model type and their number of total parameters

The +2 in the RNN calculation is to reflect Keras' `reset_after = True` GRU parameterization

4.3 Positional Encoding Methods

RNN models read in data sequentially as they train, learning order inherently. Transformers, however, lack this ability due to parallelization in the processing and require positional encoding methods to order the tokens. In this study, we will be showcasing four positional encoding methods to forecast our SEP events: Sinusoidal, Rotary Positional Encoding (RoPE), Zero-Init. These approaches injections the positional information differently and results in varied SEP prediction accuracy.

4.3.1 Sinusoidal Positional Encoding

Sinusoidal positional encoding was first introduced in the original proposed transformer architecture paper (Vaswani et al., 2017). It is an absolute position encoding method that uses sin and cos functions of different frequencies to encode the positional indexes of tokens in a sequence. For each token, a vector of dimension `dim_val` is given. each token vector is determined by the position they are in, in a sequence and the length of the vector. Odd and even indexes in the vector are determined by (2) and (1) respectively. Both (2) and (1) are used rather than just (1) because we can represent a linear relationship between two encoded positions. Once these tokens have their positional indexes the attention block computes their own QKV matrices allowing for predictions to occur.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}}) \tag{1}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}}) \tag{2}$$

The frequency of the sin and cos functions decreases exponentially by a scale factor as the position in the model dimensions increase. For short term we can find fine grain details in data, a sin / cos function with a high frequency. At large vector indexes the frequency greatly decreases, taking on a more linear relationship. The sin and cos functions keep all values tokenized values within a range of [-1,1] which is easy for the transformer to understand eliminating the need for standardization. The positions are encoded uniquely based on value and position, allowing it to scale to unlimited sequence lengths.

4.3.2 Rotary Positional Embedding

Rotary Positional Embedding (RoPE) is a relative positional embedding method that encodes the relative positions of embedded vectors to the dot product of two rotated vectors. The rotation of the vectors is determined by their order in the sequence. The relative positions are then used for self attention formulation (Su et al., 2021). The rotated embedded vector is the product of the rotation matrix, query-key weight matrix and our input vector:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W^{(11)}_{\{q,k\}} & W^{(12)}_{\{q,k\}} \\ W^{(21)}_{\{q,k\}} & W^{(22)}_{\{q,k\}} \end{pmatrix} \begin{pmatrix} x^{(1)}_m \\ x^{(2)}_m \end{pmatrix} \tag{3}$$

in 2D coordinates. A simple rotation of the word embedding vector by an angle multiple of its position index is the main idea behind Rotary Position Embedding (Su et al., 2021).

A property of RoPE is long term decay. This means that pairs of tokens at large relative distance will have less connection than those that are close to each other (Su et al., 2021). For our flux data, this would coincide with the idea from (Posner, 2007) that only a short term forecasting of relativistic electrons is required to accurately predict future proton storms from 30 minutes to an hour.

4.3.3 Zero-Initialized

Zero-Initialized positional encoding is an absolute positional that sets all positional encoding parameters in a transformer to zero at the start of training This means that positon of a timestamped flux has equal effects on each other at the start. then as training continues, near timestamped fluxes learn to correlate to each other rather than further ones. With a zero or near zero initialization, models on their own learn patterns that are used to make predictions with zero valued starting weights. Zero init contrast Sinusoidal who all have non zero starting weight values. RoPE does not have positional parameters rather computes weights analytically. This makes it reliable for generalization which is ideal for SEP data made almost entirely of background fluxes (Torres et al., 2025). Commonly, small near zero initializations sampled with normal distributions were found to perform better than larger ones (Ito et al., 2024).

Most timestamps in the relatively small proton electron flux dataset are near zero background values. A larger starting weights already has background information included leading to forecasting more background in the future during the early portion of training. Its not until after rounds of epochs to unlearn patterns are to finally map where spikes in fluxes start to occur in order to make accurate predictions.

4.4 Auto-Gluon

5 Experimental Evaluation

We measure teh forecasting effectiveness on its abilities to predict the proton flux and if an SEP event occurs 30 (t+6) minutes and (t+12) minutes ahead.

5.1 Evaluation of SEP Time-Intensity Profile Prediction

5.1.1 Evaluation Criteria

We compute mean absolute error (MAE) across all models and their predictions. We sum all errors at each timestep between the true versus predicted proton flux and take the mean.

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=0}^{N} |x_{i,\mathrm{pred}} - x_{i,\mathrm{true}}| \tag{4}$$

We measure the lag time of between the predicted and true proton flux. We shift our predicted protons between the onset and peak of each event align with the true protons and measure MAE at each shift. Once we have the minimum value of MAE we measure the time lag.

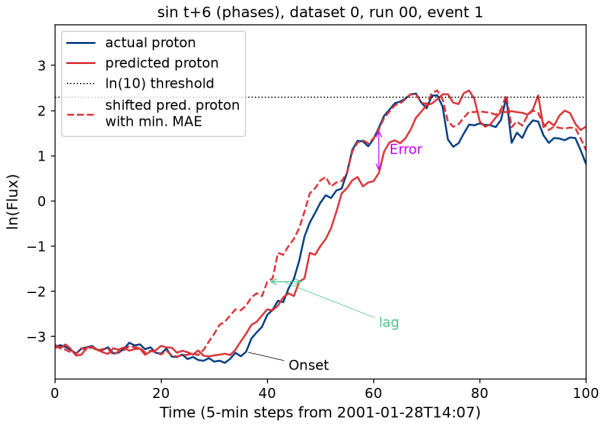


Figure 5. Lag observed on event 1 from sinusoidal transformer.

prediction efficiency metric is used as a comparison as well. For a discrete time series

$$\mathrm{PE} = 1 - \frac{\sum_{i=0}^{N} \left(x_{i,\mathrm{pred}} - x_{i,\mathrm{true}}\right)^2}{N\sigma_{\mathrm{true}}^2} \tag{5}$$

x_{true} is the true proton, x_{pred} is the predicted signal, and σ^2_{true} is the variance in the true signal. PE $= 1$ indicates perfect model performance and PE $= 0$ indicates performance comparable to predicting the arithmetic mean of the true signal. PE < 0 indicates poor predictions where guessing the predicted proton would be more effective. (Rastätter et al., 2011)

An F1 score evaluates the performance of our forecasting model. The score is calculated by arithmetic operations of the number of true positive, false positives, and false negatives the solve into the recall (6) and precision (7). the recall and precision values are then used to calculate our final F1 score (8):

$$\text{recall} = \frac{TP}{TP + FN} \tag{6}$$

$$\text{precision} = \frac{TP}{TP + FP} \tag{7}$$

$$\text{F1} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \tag{8}$$

5.1.2 Learning and Evaluation Procedures

To measure variability of our statistics we run ten blocked bootstraps experiments. A block bootstrap partitions the original data and resample the blocks in a shuffled order for a new time series dataset. A standard bootstrap samples individual timestamps of the fluxes from the original dataset causing loss of the electron proton rising patterns necessary to make predictions.

We first block from $t-36$ before the onset of each event to the end event timestamp and slice them out of the dataset. This leaves us with the background data left in the time series. We choose a block size of $N^{1/3}$ where N is the length of our eventless time series data. (Bertail & Dudek, 2024). Repeating this process nine times plus our original dataset, d_0, we are left with ten total datasets.

Each dataset is split into training and testing portions with an 80/20 ratio. d_0 contains 21 SEP events in the training split and 18 in the test split. Bootstrapping randomly switches these events from train or test portions allowing forecasting models learn from all 39 SEP events rather than 21. Each machine learning model trains 10 models on each dataset totaling 100 models for one forecasting time lead $(t + 6, t + 12)$.

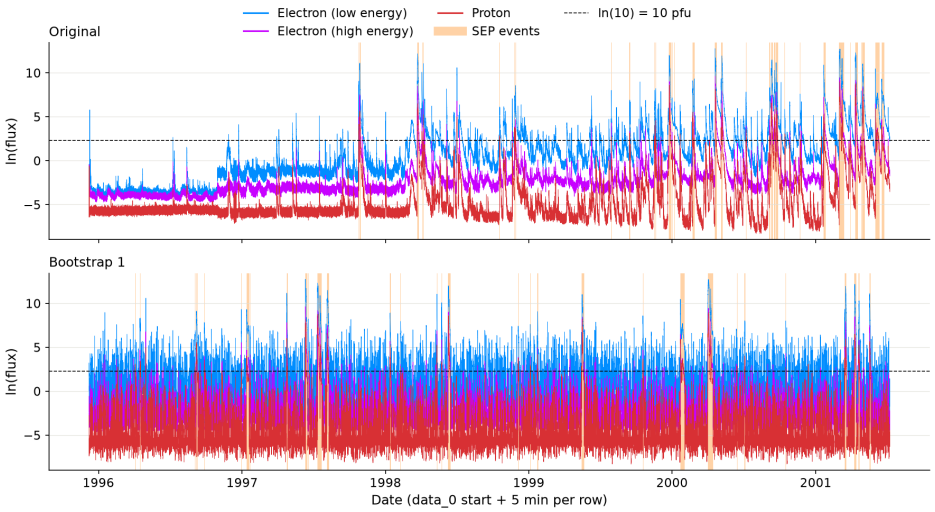


Figure 6. .

For each model and each dataset we produce a sample, θ^* of every metric (MAE, Lag, F1, PE). We assume a Student-T distribution for a small sample size ($n = 10$) and unknown population variance. We compute mean estimates for all samples to get a bootstrap estimate of a metric $\hat{\theta}$:

$$\hat{\theta}^* = \frac{1}{n}\sum_{i}^{n}\theta_i^* = \bar{\theta}^*$$

For each metric measured from a model trained on a dataset we compute the mean bootstrap estimate

$$\bar{\theta} = \frac{1}{B}\sum_{i}^{B}\hat{\theta}_i^*$$

Where B is the number of datasets. We retrieve the standard error for and the standard bias respectively:

$$S_{\hat{\theta}}^2 = \frac{1}{B-1}\sum_{i}^{B}(\hat{\theta}_i^* - \bar{\theta}^*)^2 \qquad\qquad S_b^2 = \frac{1}{B}\sum_{i}^{B}(\hat{\theta}_i^* - \bar{\theta}^*)^2$$

A bootstrap confidence is calculated with a significance level of $\alpha = .05$:

$$CI = \left(\bar{\theta} - t_{1-\alpha/2,v}\frac{s_{\theta^*}}{\sqrt{n}}, \bar{\theta} + t_{1-\alpha/2,v}\frac{s_{\theta^*}}{\sqrt{n}}\right)$$

where v is the degrees of freedom and s_{θ^*} is the sample variance of the bootrap estimate:

$$s_{\theta^*}^2 = \frac{1}{n}\sum_{i=1}^{n}(\theta_i^* - \bar{\theta}^*)^2$$

For every deep learning "champion" model we want to compare our metrics with baseline models described in Section 5.1.4. A Welch -T Test compares the mean performance of each champion model to baseline model. Neither the population variances are known making this test the most fitting method (Delacre et al., 2017). We don't expect baseline and champion models to have homogenous variances (Welch, 1947).

$$\text{t-stat} = \frac{(\bar{x}_{base} - \bar{x}_{\text{champ}}) - (\mu_{base} - \mu_{\text{champ}})}{\sqrt{\frac{s^2_{\text{base}}}{n_{\text{base}}} + \frac{s^2_{\text{champ}}}{n_{\text{champ}}}}}$$

The associated alternative hypothesis tests for each metric is as follows:

$$H_0 \text{ F1} : \mu_c \leq \mu_b \qquad H_0 \text{ PE} : \mu_c \leq \mu_b$$

$$H_0 \text{ Lag} : |\mu_c| \geq |\mu_b| \qquad H_0 \text{ MAE} : \mu_c \leq \mu_b$$

The Welch-T test does not test the variances of the sample metrics (Welch, 1947). The Levenes test has been used to measure the spread of samples (Zhou et al., 2023).

$$H_0 : \sigma^2_{\text{champ}} \geq \sigma^2_{\text{base}}$$

$$z_i = \left| \hat{\theta}^*_i - \tilde{\theta}^* \right|$$

$$\bar{z} = \frac{1}{B} \sum_i^B z_i$$

$$S^2_p = \frac{S^2_{z,\text{base}} + S^2_{z,\text{champ}}}{2}$$

$$t = \frac{\bar{z}_{\text{base}} - \bar{z}_{\text{champ}}}{S_p \sqrt{2/B}}, \qquad v = 2B - 2$$

5.1.3 Torres Machine Learning Model

To properly compare our transformer model with RNN and NN models we train the transformer to minimizes loss by the mean square error (MSE) in log space between the target proton intensity and the predicted proton intensity. The output will also be in natural logarithm space (Torres et al., 2025).

To match the RNN and NN, weights are updated using Adam optimizer and we converge if the loss function does not change by more the 10^{-4} over 20 iterations.

5.1.4 Baseline Methods Used for Comparison

A persistent model as a baseline is included in our study. its a simple model in which the predicted proton flux value is the current proton flux value, a nowcast. (Posner, 2007) Forecasting matrix will be compared to as well across the same metrics.

A multi variant linear regression model that reads in our three channels of particle fluxes will also be used as a baseline for our transformer

Because the models are detrerministic and are seedless they are only ran once per bootstrapped dataset.

6 Results

The results can be categorized into four seperate splits: (t+6, phases),(t+6, no-phases), (t+12, phases), and (t+12, no-phases) These categories are distinct and can not be compared to each other. It is only appropriate to compare models within the categories.

6.1 Bootstrap

After training and testing on each boostrap dataset we are able to obtain the behavior of a model by collecting their metrics for every bootstrap dataset.

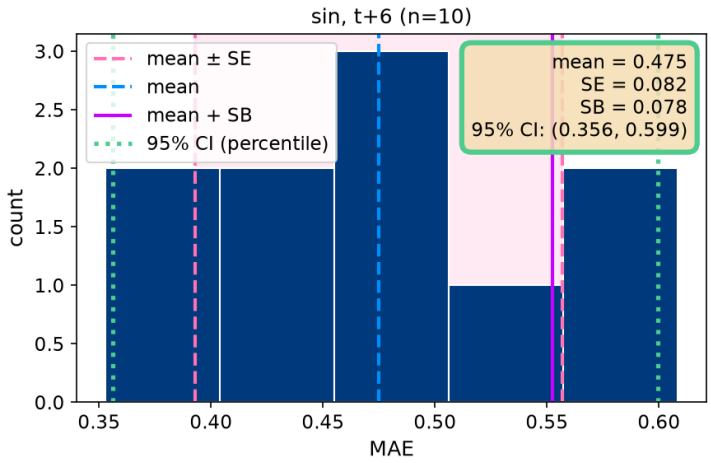


Figure 7. MAE of the sinusoidal transformer at $t + 6$ (30 minutes) on each of the 10 bootstrap datasets (seeds averaged per dataset), with the mean, mean ± SE and the 95% percentile interval.

6.1.1 Training Time

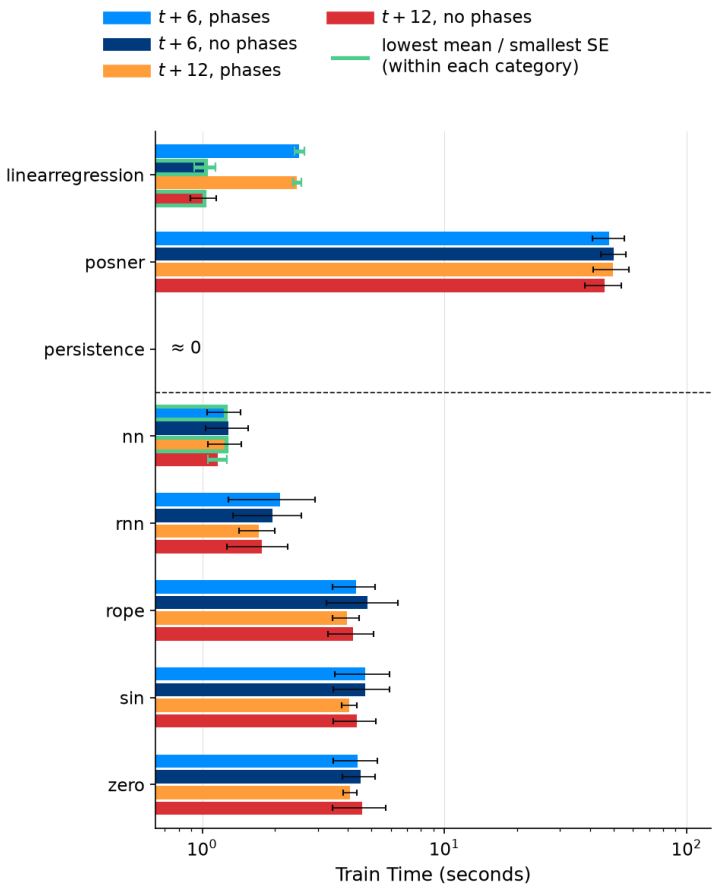


Figure 8. Mean training time of every model over the 10 bootstrap datasets, one bar per forecast horizon and phase setting, with bootstrap standard error bars. Baselines are above the dashed line and champions below it. Green marks the lowest mean (outline) and smallest standard error (error bar) within each category.

6.1.2 MAE

The mean and variance of the models metrics from every dataset tells us the effectiveness of the model predictions in a category. We can then compare the models forecast metrics with a side by side comparison.

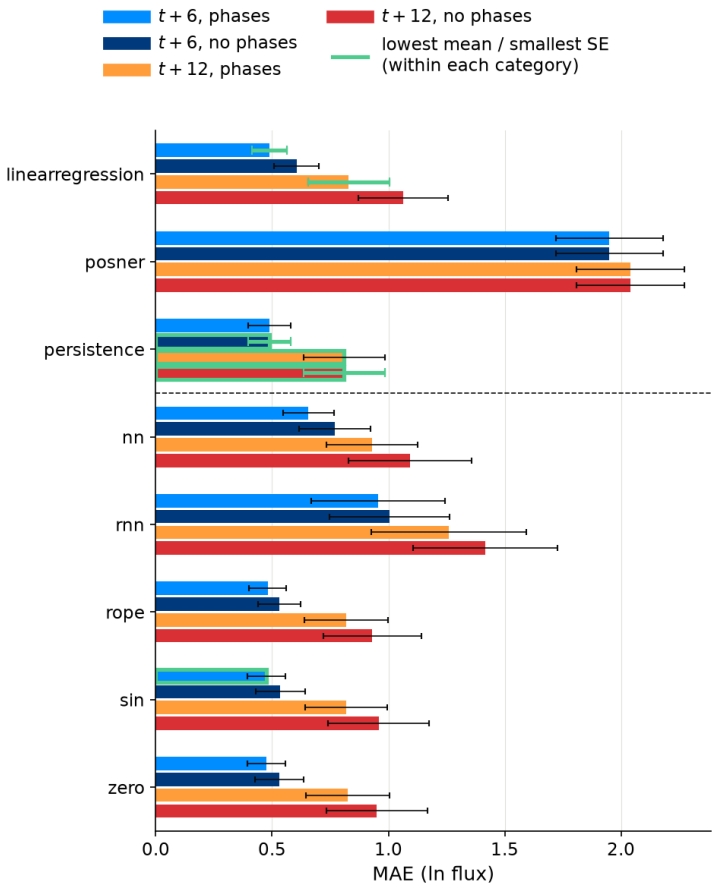


Figure 9. Mean MAE of every model over the 10 bootstrap datasets, one bar per forecast horizon and phase setting, with bootstrap standard error bars. Baselines are above the dashed line and champions below it. Green marks the lowest mean (outline) and smallest standard error (error bar) within each category.

For (t+6, phases) the model with the lowest MAE of XXXX is from model XXXXXX with a standard error of XXXXX. YYYYY had the lowest standard error of YYYYY. ($t+6$, no-phases) model with the lowest mean MAE was YYY with smallest stanard error model YYY. (t+12, phases) lowest mean MAE is ZZZ with smallest standard error ZZZ ($t+12$, no-phases) lowest mean MAE is WWW with smallest standard error WWW *Stuff here on why i think the winning models are who they are. Confirm that the NN, RNN, Posner, Persistent models matches Torres models under same parameters*

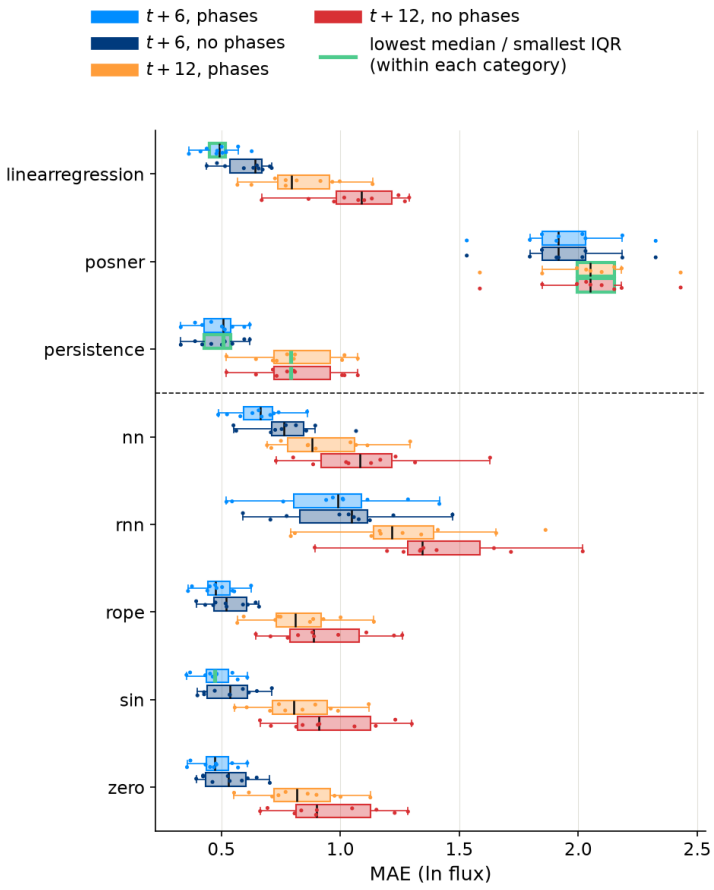


Figure 10. MAE of every model on the 10 bootstrap datasets (seeds averaged per dataset), one box per forecast horizon and phase setting. Baselines are above the dashed line and champions below it. Green marks the lowest median (line) and smallest interquartile range (box outline) within each category.

6.1.3 Lag

We duplicate the visual check with the lag values. we perform our analysis on the onset to peak lags.

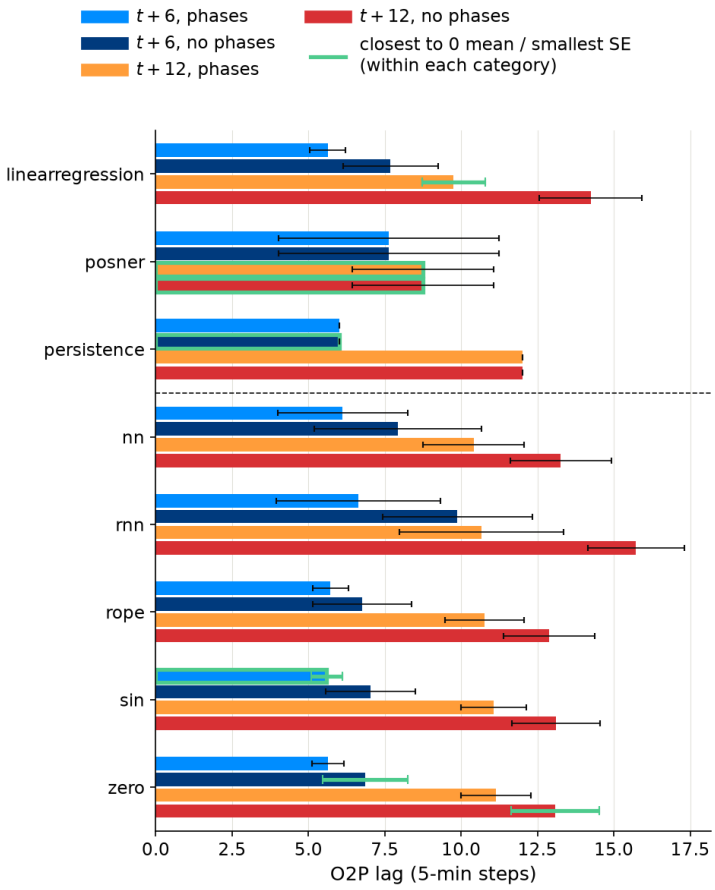


Figure 11. Mean onset-to-peak (O2P) lag of every model over the 10 bootstrap datasets, in 5-minute steps, one bar per forecast horizon and phase setting, with bootstrap standard error bars. Baselines are above the dashed line and champions below it. Green marks the mean closest to zero (outline) and smallest standard error (error bar) within each category

Posner overpredicts the rise in forecast with values $LLL{\pm}LL$. Persistence will always have an O2P lag of 6.0 and 12.0 for $(t+6, t+12)$ respectively, regardless of the flux dataset. This means that the spread will always be zero. For (t+6, phases) the model with the lowest O2P Lag of XXXX is from model XXXXXX with a standard error of XXXXX. YYYYY had the lowest standard error of YYYYY. (t+6, no-phases) model with the lowest mean O2P Lag was YYY with smallest stanard error model YYY. (t+12, phases) lowest mean O2P Lag is ZZZ with smallest standard error ZZZ (t+12, no-phases) lowest mean O2P Lag is WWW with smallest standard error WWW *Stuff here on why i think the winning models are who they are. Confirm that the NN, RNN, Posner, Persistent models matches Torres models under same parameters*



Figure 12. Onset-to-peak (O2P) lag of every model on the 10 bootstrap datasets (seeds averaged per dataset), in 5-minute steps, one box per forecast horizon and phase setting. Baselines are above the dashed line and champions below it. Green marks the median closest to zero (line) and smallest interquartile range (box outline) within each category.

6.1.4 Correlation

We repeat the comparison with the prediction efficiency (PE), where higher is better and a PE of 1 is a perfect forecast.

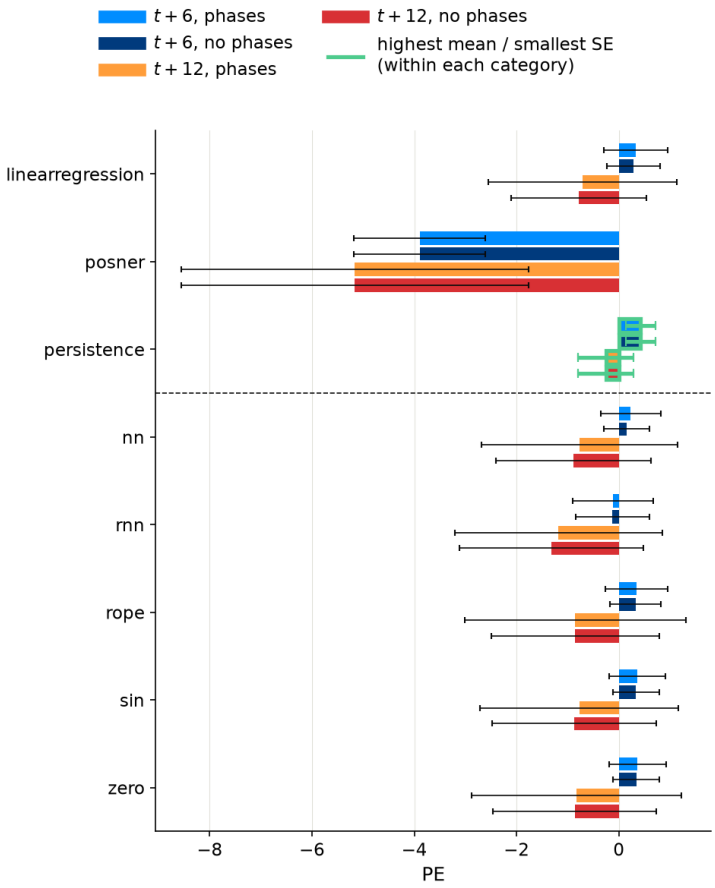


Figure 13. Mean prediction efficiency (PE) of every model over the 10 bootstrap datasets, one bar per forecast horizon and phase setting, with bootstrap standard error bars. Baselines are above the dashed line and champions below it. Green marks the highest mean (outline) and smallest standard error (error bar) within each category.

For (t+6, phases) the model with the highest PE of XXXX is from model XXXXXX with a standard error of XXXXX. YYYYY had the lowest standard error of YYYYY. (t+6, no-phases) model with the highest mean PE was YYY with smallest stanard error model YYY. (t+12, phases) highest mean PE is ZZZ with smallest standard error ZZZ (t+12, no-phases) highest mean PE is WWW with smallest standard error WWW *Stuff here on why i think the winning models are who they are. Confirm that the NN, RNN, Posner, Persistent models matches Torres models under same parameters*

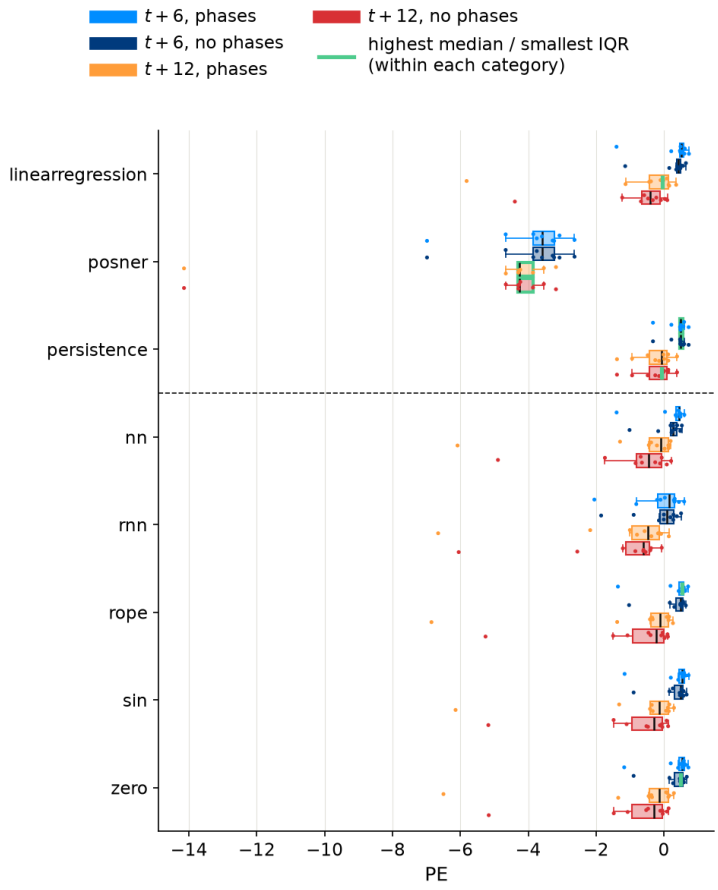


Figure 14. Prediction efficiency (PE) of every model on the 10 bootstrap datasets (seeds averaged per dataset), one box per forecast horizon and phase setting. Baselines are above the dashed line and champions below it. Green marks the highest median (line) and smallest interquartile range (box outline) within each category.

6.1.5 True vs Predicted

For each event we regress the true proton flux on each model's predicted flux, from onset to peak, and count which model has the highest R^2. The fit is $\hat{y}_i = \beta_0 + \beta_1 x_i$, with x_i the predicted and y_i the true proton flux at time step i, and the 95% prediction interval around it is

$$\mathrm{PI}_i = \hat{y}_i \pm t_{1-\alpha/2,\,N-2} \sqrt{s^2 \left(1 + \frac{1}{N} + \frac{(x_i - \bar{x})^2}{S_{xx}}\right)}$$

$$s^2 = \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{N-2}, \qquad S_{xx} = \sum_{i=1}^{N} (x_i - \bar{x})^2$$

where N is the number of time steps from onset to peak and $\bar{x}$ is the mean predicted flux. A new true value is expected to fall inside the interval 95% of the time.

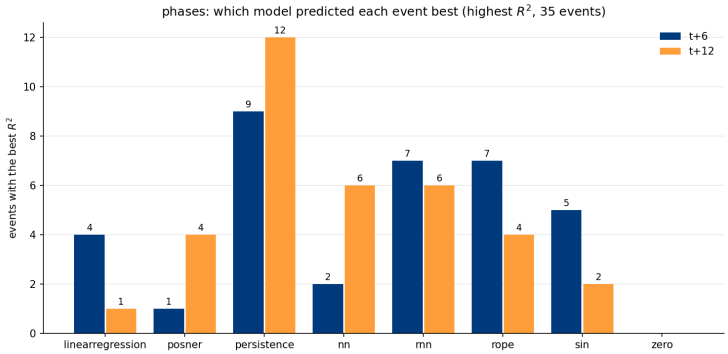


Figure 15. Number of events on which each model had the highest R^2 between true and predicted proton flux (onset to peak), with phases, at $t + 6$ and $t + 12$.

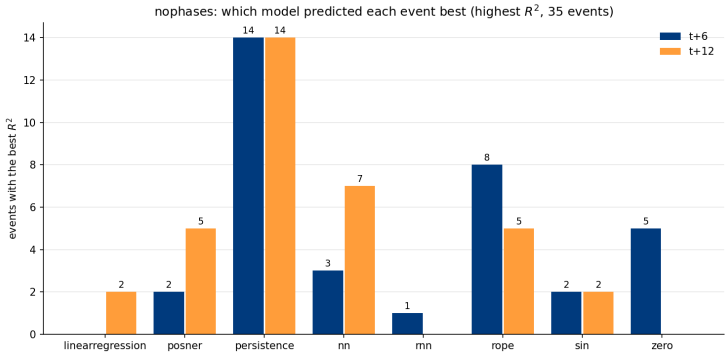


Figure 16. Number of events on which each model had the highest R^2 between true and predicted proton flux (onset to peak), without phases, at $t + 6$ and $t + 12$.

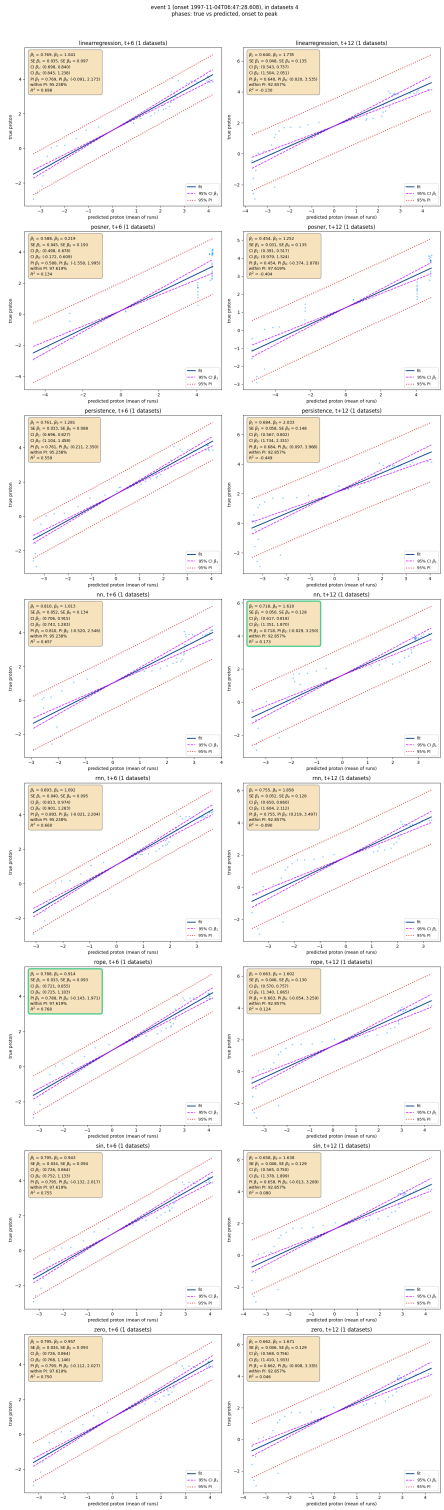


Figure 17. True vs predicted proton flux for event 1 (onset 1997-11-04), with phases: one panel per model (rows) and forecast horizon (columns), with the regression fit, the 95% confidence interval of the slope and the 95% prediction interval. Green boxes mark the highest R^2 for each horizon.

6.1.6 F1

We score each model as an alert system following (Torres et al., 2025). An alert begins when the predicted proton flux reaches ln(10) (10 pfu) and stays on until an alert window has passed after the last prediction above the threshold. An event caught by an alert that covers its threshold crossing is a true positive, an event with no such alert is a false negative, and an alert that touches no event is a false positive. The four approaches differ in when an alert starts and how it is extended: W starts the alert at the forecast target time and extends it only while consecutive predictions stay above the threshold, EW also extends it whenever another prediction above the threshold arrives within the alert window, AW and EAW are W and EW with the alert starting when the forecast is issued, t steps earlier.

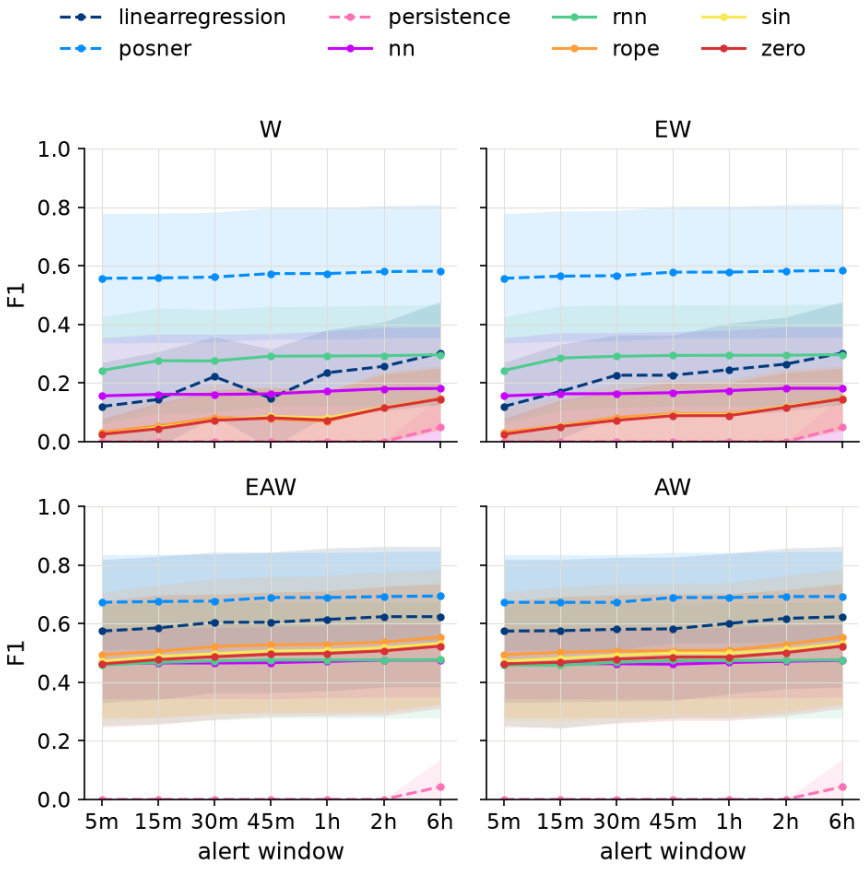


Figure 18. F1 against alert window for the four alerting approaches (W, EW, EAW, AW), ($t + 6$, phases). Each line is the mean over the 10 bootstrap datasets (seeds averaged per dataset) with a ± one standard deviation band. Baselines are dashed and champions solid.

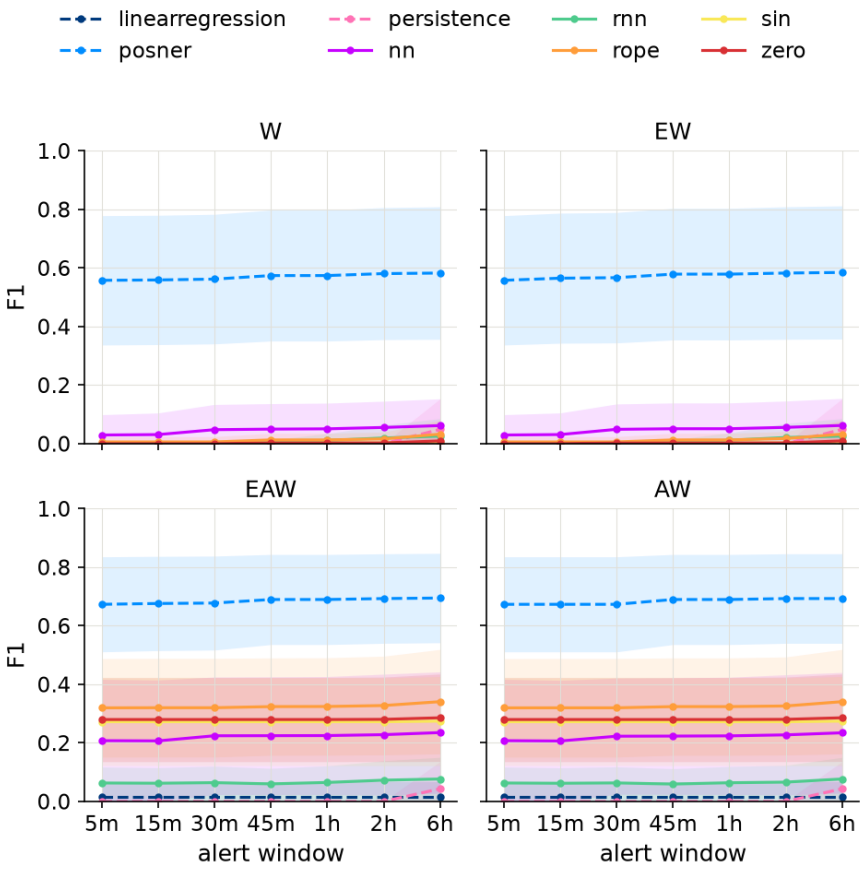


Figure 19. F1 against alert window for the four alerting approaches (W, EW, EAW, AW), (t + 6, no-phases). Each line is the mean over the 10 bootstrap datasets (seeds averaged per dataset) with a ± one standard deviation band. Baselines are dashed and champions solid.

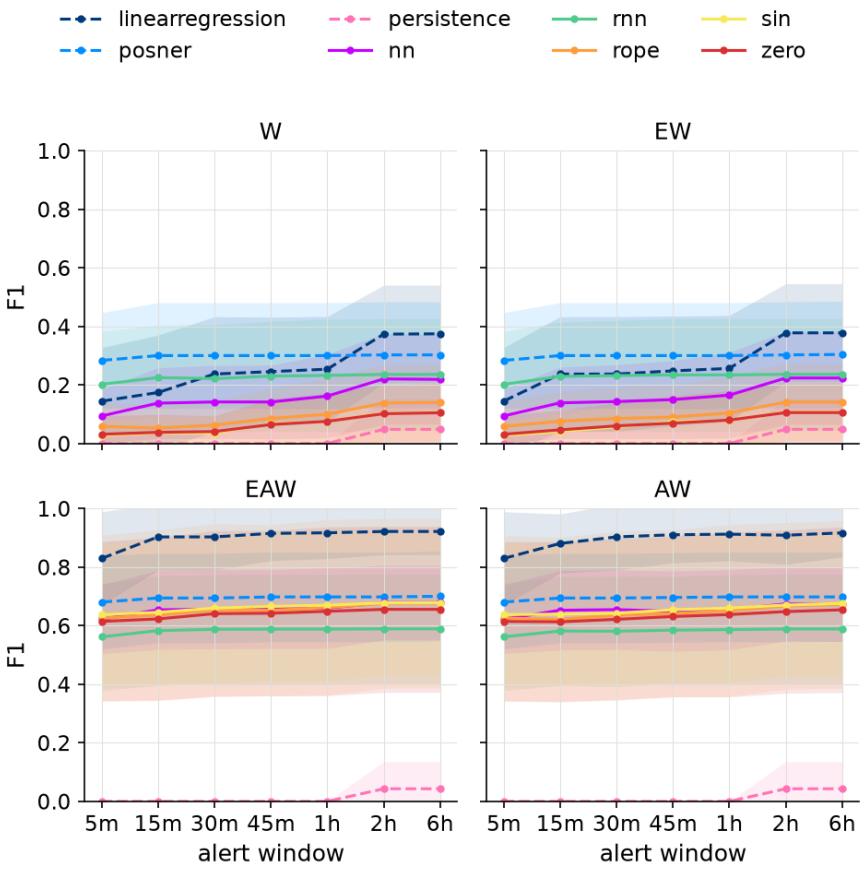


Figure 20. F1 against alert window for the four alerting approaches (W, EW, EAW, AW), (t+12, phases). Each line is the mean over the 10 bootstrap datasets (seeds averaged per dataset) with a ± one standard deviation band. Baselines are dashed and champions solid.

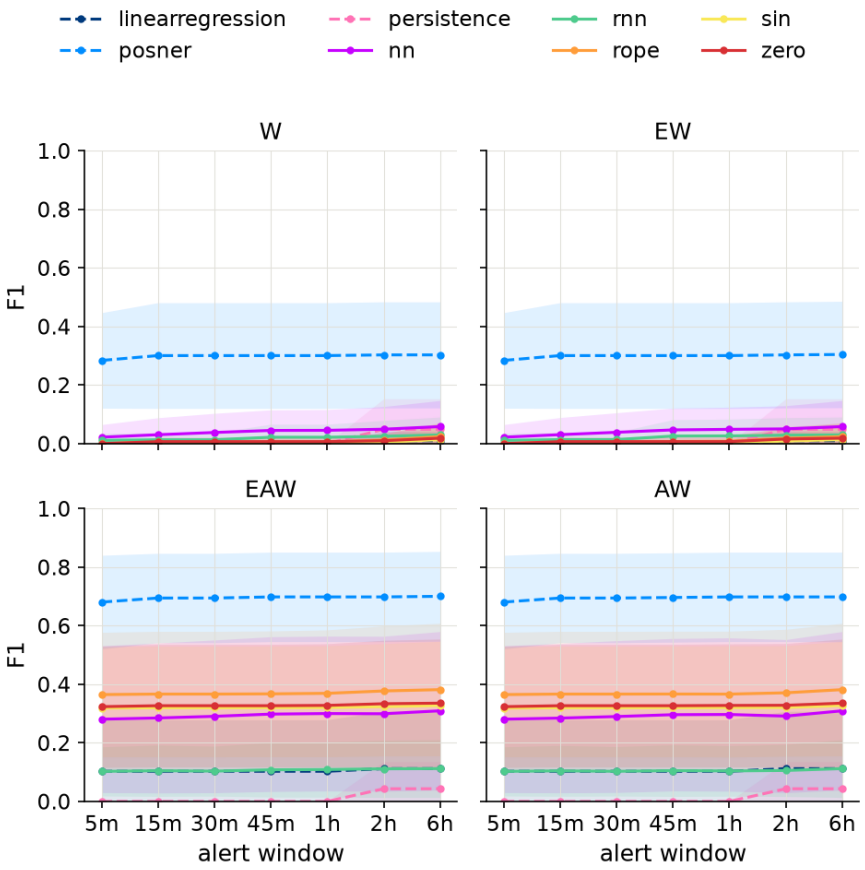


Figure 21. F1 against alert window for the four alerting approaches (W, EW, EAW, AW), (t + 12, no-phases). Each line is the mean over the 10 bootstrap datasets (seeds averaged per dataset) with a ± one standard deviation band. Baselines are dashed and champions solid.

For (t+6, phases) the highest F1 of XXXX is from model XXXXXX with approach XXX at an alert window of XXX. (t+6, no-phases) the highest F1 was YYY with approach YYY. (t + 12, phases) the highest F1 is ZZZ with approach ZZZ. (t + 12, no-phases) the highest F1 is WWW with approach WWW. *Stuff here on why i think the winning models and approaches are who they are, and how the F1 changes with the alert window.*

6.2 Hypothesis Testing

We test whether each champion is better than each baseline within every category, using the one-sided Welch's t-test (Welch, 1947) on the per-dataset means and the one-sided Levene's test with the median centre (Zhou et al., 2023) on the per-dataset spread,

both at $\alpha = 0.05$. For the lags the Welch's t-test compares the distance from zero, |lag|. Rejecting H_0 means the champion has the better mean (Welch) or the smaller spread (Levene). Below are the results for the sinusoidal transformer.



Figure 22. One-sided Welch's t-test of the sinusoidal transformer against each baseline, for every metric and category, on the per-dataset means. Green: H_0 rejected (sinusoidal transformer better); red: not rejected. Each cell gives the one-sided p-value.

Against XXXX the sinusoidal transformer has a significantly better mean in XXXX, while against YYYY it is not significantly better in any metric except YYYY. *Stuff here on why the sinusoidal transformer does or does not beat each baseline.*

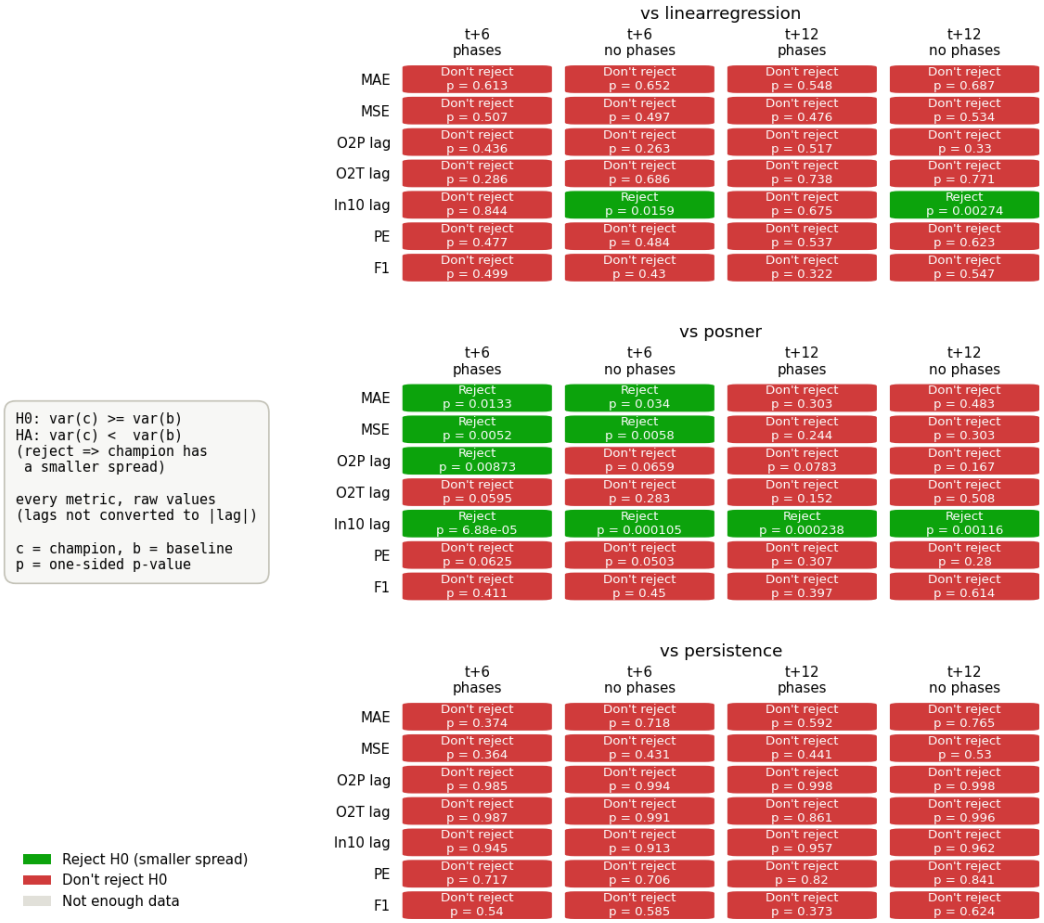


Figure 23. One-sided Levene's test (median centre) of the sinusoidal transformer against each baseline, for every metric and category, on the per-dataset values. Green: H_0 rejected (sinusoidal transformer has a smaller spread); red: not rejected. Each cell gives the one-sided p-value.

Against XXXX the sinusoidal transformer has a significantly smaller spread in XXXX, while against YYYY it is not significantly tighter in any metric except YYYY. *Stuff here on why the spread of the sinusoidal transformer is or is not smaller than each baseline.*

7 Conclusion

Our goal was to compare the forecasting of a RNN of relativistic electrons to be able to predict incoming protons thrity minutes to an hour later. Using multiple positional encoding methods we were able to compare our model to a regular NN and RNN model performing the same task

Overall, our results indicate better reduction in MAE but relatively similar lag and F1 scores, however highly outperforming the forecasting matrix from (Posner, 2007). Mirroring (Torres et al., 2025), transformers have issues with delyaed SEP proton predictions whic leads to yielding poorer recall, precision, and therefore F1 scores. scaling these models with more electron inputs would lead to higher proton prediciton accuracy.

7.1 Future Work

After a baseline comparison of Torres RNN model, a hyper parameterization search can be used to find the model with the highest prediction accuracy. Other inputs to further inform the model such as coronal mass ejections (CME) or magnetic B fields could also yield a higher prediction accuracy and F1 score. Further investigation of the effects of space weather into the the satellites components is another possibility. This requires satellite specific models to be produced to take in Single Event Upset (SEU) as an input. The main goal is to transfer techiniques developed here towards LEO sats such as GOES with there proton flux data.

Acknowledgments

TBD.

Funding

TBD.

Open Research Statement

TBD.

Conflict of Interest

TBD.

References

Barak, J., Reed, R. A., & LaBel, K. A. (1999, December). On the figure of merit model for SEU rate calculations. *IEEE Transactions on Nuclear Science*, *46*(6), 1504–1510.

Bertail, P., & Dudek, A. E. (2024). Optimal choice of bootstrap block length for periodically correlated time series. *Bernoulli*, *30*(3), 2521–2545. doi: 10.3150/23-BEJ1683

Cai, L., Janowicz, K., Mai, G., Yan, B., & Zhu, R. (2020). Traffic transformer: Capturing the continuity and periodicity of time series for traffic forecasting. *Transactions in GIS*, *24*(3), 736–755. doi: 10.1111/tgis.12644

Carlton, A., Morgan, R., Lohmeyer, W., & Cahoy, K. (2018). Telemetry fault-detection algorithms: Applications for spacecraft monitoring and space environment sensing. *Journal of Aerospace Information Systems*, *15*(5), 239–252. doi: 10.2514/1.I010587

Delacre, M., Lakens, D., & Leys, C. (2017). Why psychologists should by default use Welch's t-test instead of Student's t-test. *International Review of Social Psychology*, *30*(1), 92–101. doi: 10.5334/irsp.82

Ito, T., Cocchi, L., Klinger, T., Ram, P., Campbell, M., & Hearne, L. (2024). Learning interpretable positional encodings in transformers depends on initialization. *arXiv preprint arXiv:2406.08272*. doi: 10.48550/arXiv.2406.08272

Müller-Mellin, R., Kunow, H., Fleißner, V., Pehlke, E., Rode, E., Röschmann, N., ... Henrion, J. (1995). COSTEP – comprehensive suprathermal and energetic particle analyser. *Solar Physics*, *162*(1-2), 483–504. doi: 10.1007/BF00733437

Noeldeke, C., Boettcher, M., Mohr, U., Gaisser, S., Alvarez Rua, M., Eickhoff, J., ... Varatharajoo, R. (2021). Single event upset investigations on the "flying laptop" satellite mission. *Acta Astronautica*. doi: 10.1016/j.actaastro.2020.12.052

Posner, A. (2007). Up to 1-hour forecasting of radiation hazards from solar energetic ion events with relativistic electrons. *Space Weather*, *5*(5). doi: 10.1029/2006SW000268

Rastätter, L., Kuznetsova, M. M., Vapirev, A., Ridley, A., Wiltberger, M., Pulkkinen, A., ... Singer, H. J. (2011). Geospace environment modeling 2008–2009 challenge: Geosynchronous magnetic field. *Space Weather*, *9*(4), S04005. doi:

10.1029/2010SW000617

Soho comprehensive suprathermal and energetic particle analyzer (costep) electron proton helium instrument (ephin) intensity level 3 1-minute data in cdf format. (2026). Retrieved from `https://spase-metadata.org/ESA/NumericalData/SOHO/COSTEP/EPHIN/CDF/PT1M`

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*. doi: 10.48550/arXiv.2104.09864

Torres, J., Chan, P. K., Zhao, L., & Zhang, M. (2025). A machine learning approach to predicting sep proton intensity and events using time series of relativistic electron measurements. *Space Weather*. doi: 10.1029/2024SW003921

Van Hollebeke, M. A. I., Ma Sung, L. S., & McDonald, F. B. (1975). The variation of solar proton energy spectra and size distribution with heliolongitude. *Solar Physics*, *41*, 189–223. doi: 10.1007/BF00152967

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (Vol. 30, pp. 5998–6008).

Welch, B. L. (1947). The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, *34*(1–2), 28–35. doi: 10.1093/biomet/34.1-2.28

Zhao, J., Chu, F., Xie, L., Che, Y., Wu, Y., & Burke, A. F. (2026). A survey of transformer networks for time series forecasting. *Computer Science Review*, *60*, 100883. doi: 10.1016/j.cosrev.2025.100883

Zhou, Y., Zhu, Y., & Wong, W. K. (2023). Statistical tests for homogeneity of variance for clinical trials and recommendations. *Contemporary Clinical Trials Communications*, *33*, 101119. doi: 10.1016/j.conctc.2023.101119